Projektarbeit

Fachbereich M – Master Maschienenbau Mechatraonik | WS 2025/2026

Entwicklung, Integration und Validierung einer ROS2-basierten Hinderniserkennung mit Reaktionsstrategie für automatisierte Modellfahrzeuge

Autoren:	Aly Elbermawy (5335848) Omar Elalami (5379642) Zakaria Elaboudi (5223183)
Betreuer:	Prof. Dr. Benedikt Lattke
Abgabedatum:	12. April 2026

Kurzfassung

Diese Projektarbeit beschreibt die Integration, Dokumentation und Bewertung eines scan-basierten *Follow-the-Gap*-Stacks (FTG) für das MXCarkit. Das Fahrzeug basiert auf einem NVIDIA-Jetson-System, arbeitet mit einem 2D-LiDAR und wird über eine Ackermann-Steuerkette mit VESC angesteuert. Ziel der Arbeit war es, einen robusten, nachvollziehbaren und sicher testbaren ROS 2-Datenpfad von `/scan` bis `/autonomous/ackermann_cmd` aufzubauen.

Die aktualisierte Zielarchitektur besteht aus vier logisch getrennten Stufen: (i) einer TF-basierten Vorverarbeitung des LiDAR-Scans, (ii) dem C++-Planerkern follow_the_gap_v0 im Scan-Modus, (iii) einem Python-Planer für Lenkwinkel- und Geschwindigkeitsvorgabe und (iv) einem Ackermann-Command-Knoten als Ausgabe in die bestehende MXCK-Steuerkette. Der bisherige Legacy-Pfad über obstacle substitution bleibt kompatibel im Repository, ist jedoch nicht mehr der primäre Laufzeitpfad.

Die theoretische Einordnung des Verfahrens erfolgt anhand der Originalarbeit von Sezer und Gokasan zum Follow-the-Gap-Verfahren, der CTU-Implementierung für F1/10-Fahrzeuge sowie der ROS 2/F1TENTH-Migrationsarbeit von Endler. Auf dieser Basis wird gezeigt, wie sich die klassische FTG-Idee (largest gap, gap center, final heading) in eine aktuelle ROS 2-Pipeline für ein Jetson-basiertes Lehr- und Versuchssystem übertragen lässt.

Inhaltsverzeichnis

Kurzfassung **II**

Inhaltsverzeichnis **III**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

1 Grundlagen **1**

1.1 Aufgabenstellung und Ziel der Arbeit 1

1.2 Das MXCarkit als Zielplattform 1

1.3 Grundlagen von ROS 2 3

1.3.1 Workspace 3

1.3.2 Paket 4

1.3.3 Node 4

1.3.4 Launch-Dateien und Parameter 4

1.4 Stand der Technik 5

1.4.1 Vergleich bestehender Planungsansätze 5

1.4.2 Sensorik zur Umfeldwahrnehmung 5

1.5 Follow-the-Gap als reaktiver Planungsansatz 6

1.5.1 Grundprinzip und methodisches Vorgehen 6

1.5.2 Eigenschaften des Verfahrens 7

1.5.3 Scan-basierte Umsetzung im ROS 2-Kontext 7

2 Systembeschreibung und Durchführung **8**

2.1 Gesamtarchitektur des Systems 8

2.2 Algorithmische Grundlage 9

2.3 Stufe 1: LiDAR-Vorverarbeitung 10

2.3.1 Funktionale Aufgaben 10

2.3.2 Node Perception Erklärung 11

2.4 Stufe 2: Follow-the-Gap-Kern 11

2.4.1 Verarbeitungsschritte 11

2.4.2 Ausgabegrößen 11
2.5 Stufe 3: Planner 11
2.5.1 Funktionale Rolle 11
2.5.2 Interpretation der Sensordaten 12
2.6 Stufe 4: Control 12
2.6.1 Aufgabe 12
2.6.2 Systemintegration 12
2.7 Versuchsdurchführung 12
2.7.1 Versuchsaufbau 13
2.7.2 Parametrierung des Systems 14
2.7.3 Messmethodik und Datenerfassung 17

3 Diskussion und Bewertung 21
3.1 Bewertung der gewählten Gesamtarchitektur 21
3.1.1 Reduktion von Komplexität 21
3.1.2 Vergleich mit alternativen Ansätzen 21
3.2 Analyse der Planner- und Control-Schicht 22
3.3 Bewertung der experimentellen Ergebnisse 22
3.3.1 Quantitativer Vergleich der Validierungsläufe 23
3.3.2 Zeitreihenbasierte Detailanalyse 23
3.3.3 Bewertung der Hinderniserkennung 25
3.3.4 Bewertung der Ausweichmanöver 26
3.3.5 Einfluss der Parametrierung auf das Fahrverhalten 27
3.3.6 Abgleich mit den Projektanforderungen 28
3.3.7 Empfehlungen aus der Validierung 28
3.3.8 Robustheit gegenüber statischen Hindernissen 29
3.3.9 Grenzfälle und Grenzen des Systems 29
3.4 Einordnung in den wissenschaftlichen Kontext 30

4 Fazit und Ausblick 31
4.1 Fazit 31
4.2 Ausblick 32
Schlussbemerkung 33

Literaturverzeichnis 34

Abbildungsverzeichnis

1.1 Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera. 2
1.2 Systemarchitektur und Hardware-Komponenten des MXCarkit [8]. . . . 3

2.1 Gesamtarchitektur des scan-basierten FTG-Stacks . Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die klare Trennung zwischen Wahrnehmung, Planung und Steuerung.[7] 9
2.2 Schematische Darstellung des Follow-the-Gap-Prinzips [6]. 10
2.3 Versuchsaufbau Aufnahme 1: Korridor mit einem statischen Hindernis. 13
2.4 Versuchsaufbau Aufnahme 2: Mehrere Hindernisse und dynamische Veränderung der Umgebung. 13
2.5 Exemplarisches Foxglove-Layout zur Online-Beobachtung des FTG-Stacks während der Validierungsfahrten. Dargestellt sind Sensorik, Visualisierungstopics und zentrale Zustandsgrößen des Planungs- und Steuerpfads. 20

3.1 Zeitreihenbasierte Auswertung von Validierungslauf A. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners. 24
3.2 Zeitreihenbasierte Auswertung von Validierungslauf B. Die Darstellung verdeutlicht Unterschiede in Gap-Erkennung, Geschwindigkeitsfreigabe und Planner-Zuständen im Vergleich zu Validierungslauf A. 25

Tabellenverzeichnis

2.1 Verwendete Laufzeitparameter des FTG-Stacks während der Validierungsfahrten. 14
2.2 Interne FTG-Konstanten des C++-Kerns `follow_the_gap_v0`. 17
2.3 Für die Validierung aufgezeichnete Topics des FTG-Stacks. 19

3.1 Vergleich zentraler Metriken der beiden Validierungsläufe. 23
3.2 Abgleich der wesentlichen Projektanforderungen mit den Validierungsergebnissen. 28

1 Grundlagen

Dieses Kapitel legt die fachlichen und systemtechnischen Grundlagen der vorliegenden Arbeit dar. Zunächst wird die Aufgabenstellung präzise eingeordnet und das Zielsystem beschrieben. Anschließend werden die relevanten Konzepte von ROS 2 erläutert, die für das Verständnis der implementierten Architektur notwendig sind. Abschließend wird der Follow-the-Gap-Algorithmus sowohl in seiner ursprünglichen Form als auch in der für diese Arbeit relevanten scan-basierten Ausprägung dargestellt.

1.1 Aufgabenstellung und Ziel der Arbeit

Im Rahmen dieser Arbeit wird eine Fahrfunktion zur Hinderniserkennung für ein automatisiertes Modellfahrzeug auf Basis des MxCarkit-Systems entwickelt. Ziel ist es, eine robuste und echtzeitfähige Lösung zu konzipieren, die Hindernisse im Fahrweg zuverlässig erkennt und darauf mit geeigneten Reaktionsstrategien wie Abbremsen oder Ausweichen reagiert.

Die Aufgabenstellung umfasst die Entwicklung, Integration und Validierung einer ROS 2-basierten Softwarearchitektur, welche Sensordaten, insbesondere von LiDAR- oder Kamerasystemen, verarbeitet und daraus sichere Fahrentscheidungen ableitet. Dabei sollen sowohl geeignete Verfahren zur Umfeldwahrnehmung als auch eine darauf abgestimmte Reaktionslogik implementiert und experimentell untersucht werden.

Ein weiterer Schwerpunkt liegt auf der Systemintegration in die bestehende Fahrzeugarchitektur sowie der experimentellen Validierung unter Laborbedingungen. Abschließend erfolgt eine Bewertung der Leistungsfähigkeit, Robustheit und Grenzen des entwickelten Systems, um dessen Einsatzpotenzial für autonome Fahrfunktionen einzuordnen.

1.2 Das MXCarkit als Zielplattform

Das MXCarkit (MXCK) stellt die physische und softwareseitige Zielplattform dieser Arbeit dar. Es handelt sich um ein Jetson-basiertes Versuchsfahrzeug, das für Lehre und Entwicklung im Bereich autonomer Fahrfunktionen konzipiert ist.

Abbildung 1.1 zeigt den physischen Aufbau des Versuchsfahrzeugs mit den für diese Arbeit zentralen Sensorkomponenten.

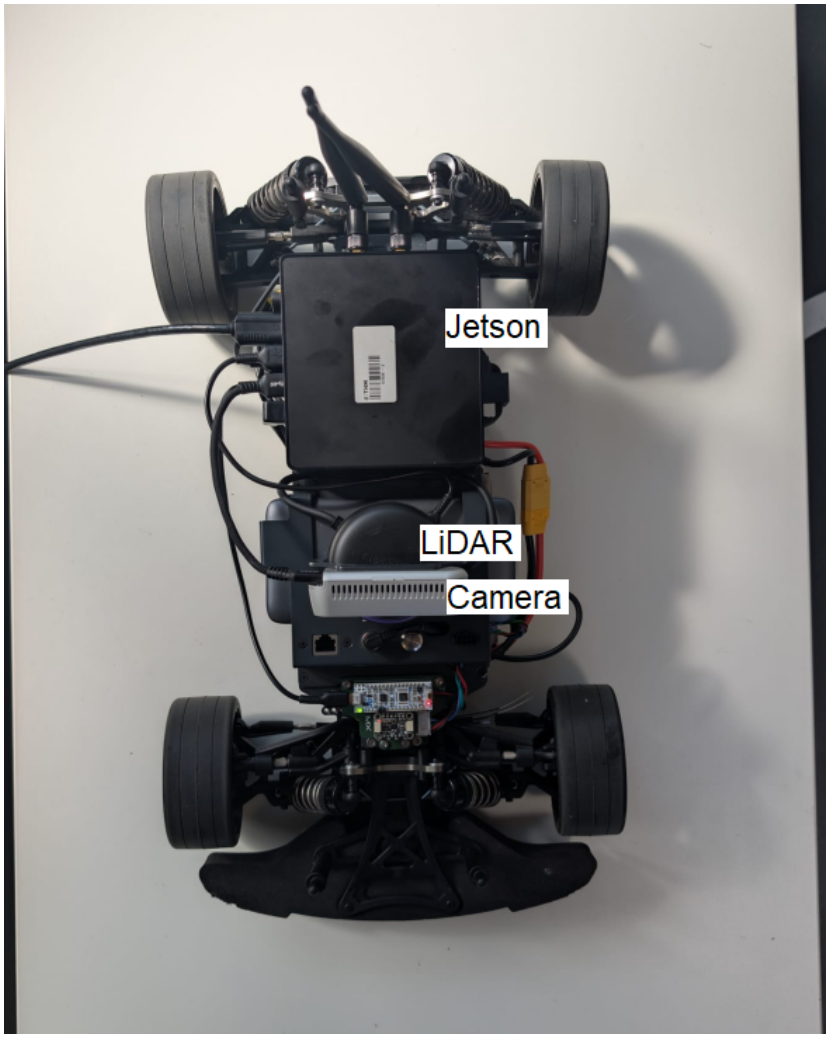

Abbildung 1.1: Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera.

Die Systemarchitektur des MXCK ist modular aufgebaut und nutzt eine containerisierte Umgebung. Eine detaillierte Übersicht der Hardware-Komponenten und deren Vernetzung ist in Abbildung 1.2 dargestellt.

Abbildung 1.2: Systemarchitektur und Hardware-Komponenten des MXCarkit [8].

Zentrale Funktionalitäten sind in separaten Workspaces organisiert, insbesondere:

- `mxck2_ws` für die Kernsysteme (Sensorik, Steuerung),
- `development_ws` für experimentelle Erweiterungen.

1.3 Grundlagen von ROS 2

Da die gesamte Systemarchitektur auf ROS 2 basiert, ist ein präzises Verständnis der zugrunde liegenden Konzepte erforderlich.

1.3.1 Workspace

Ein ROS 2-Workspace ist ein Verzeichnis, in dem mehrere Pakete gemeinsam verwaltet und gebaut werden. Die typische Struktur umfasst die Verzeichnisse `src/`, `build/`, `install/` und `log/`. Der Build-Prozess erfolgt üblicherweise über `colcon`.

Im Kontext des MXCK wird diese Struktur direkt genutzt, wobei unterschiedliche Workspaces verschiedene Verantwortlichkeiten abdecken.

1.3.2 Paket

Ein Paket stellt die zentrale organisatorische Einheit in ROS 2 dar. Es bündelt eine klar abgegrenzte Funktionalität, beispielsweise im Bereich Perception, Planung oder Steuerung.

Je nach Implementierungssprache unterscheiden sich die notwendigen Dateien:

- Python: `package.xml`, `setup.py`, `setup.cfg`,
- C++: `package.xml`, `CMakeLists.txt`.

Zusätzlich können Launch-Dateien und YAML-Konfigurationen Bestandteil eines Pakets sein.

1.3.3 Node

Ein Node ist ein eigenständiger Prozess innerhalb des ROS 2-Systems. Nodes kommunizieren über Topics, Services oder Actions und bilden gemeinsam einen verteilten Verarbeitungsgraphen.

Im vorliegenden System entspricht jede funktionale Stufe des FTG-Stacks einem eigenen Node, was eine klare Trennung der Verantwortlichkeiten ermöglicht.

1.3.4 Launch-Dateien und Parameter

ROS 2 verwendet bevorzugt Python-basierte Launch-Dateien zur Orchestrierung mehrerer Nodes. Diese definieren:

- welche Nodes gestartet werden,
- welche Parameter geladen werden,
- sowie die verwendeten Topic-Namen und Namespaces.

YAML-Dateien dienen dabei der strukturierten Parametrierung der Nodes. Für die Reproduzierbarkeit des Systems ist die Konsistenz zwischen Launch-Dateien, Parametern und tatsächlicher Node-Konfiguration entscheidend.

1.4 Stand der Technik

Die autonome Navigation von Robotern wird in der Fachliteratur klassisch in die globale Pfadplanung und die lokale Hindernisvermeidung unterteilt. Während statische Planer eine vollständige Karte der Umgebung benötigen, agiert die Hindernisvermeidung rein reaktiv auf Basis aktueller Sensordaten.

1.4.1 Vergleich bestehender Planungsansätze

In der Entwicklung haben sich verschiedene Ansätze zur lokalen Navigation etabliert, die jedoch spezifische systemtechnische Nachteile aufweisen:

- **Bug-Algorithmen**: Diese folgen dem Prinzip, sich direkt auf ein Ziel zuzubewegen und bei Kontakt ein Hindernis am Rand zu umfahren. Dies führt jedoch oft zu ineffizienten Trajektorien und einer gefährlich nahen Annäherung an Objekte.
- **Artificial Potential Fields (APF)**: Hierbei werden Hindernisse als abstoßende und das Ziel als anziehende Potentiale modelliert. Ein kritisches Problem stellt das Auftreten lokaler Minima dar, bei denen sich die wirkenden Kräfte gegenseitig aufheben und der Roboter vor Erreichen des Ziels verharrt [1].
- **Vector Field Histogram (VFH)**: Dieser Ansatz nutzt ein polares Histogramm zur Richtungsbestimmung. Er ist zwar zielorientiert, vernachlässigt jedoch häufig die spezifischen kinematischen Beschränkungen des Fahrzeugs, was zu nicht fahrbaren Pfaden führen kann.

Aufgrund dieser Defizite wurde für die vorliegende Arbeit die *Follow-the-Gap Method* (FGM) gewählt. Im Gegensatz zu APF-Methoden ist FGM mathematisch resistent gegen lokale Minima, da der Algorithmus gezielt nach dem größten freien Bereich sucht, anstatt lediglich Kraftvektoren zu summieren [1]. Zudem integriert FGM die kinematischen Beschränkungen (Non-Holonomie) des Fahrzeugs direkt in den Auswahlprozess der Lücke, was eine höhere Trajektoriengüte und Sicherheit gewährleistet.

1.4.2 Sensorik zur Umfeldwahrnehmung

Für die Umsetzung einer robusten Hindernisvermeidung ist die Wahl der Sensorik entscheidend. In dieser Arbeit wird ein 2D-LiDAR-System (Light Detection and Ranging) als Primärsensor eingesetzt. Die Entscheidung für LiDAR gegenüber kamerabasierten Systemen (Vision) basiert auf folgenden wissenschaftlichen Kriterien:

1. **Präzision und Reichweite**: LiDAR liefert direkte und hochpräzise Distanzmessungen mit einer geringen Latenz, was für reaktive Algorithmen wie FGM essentiell ist [1].

2. **Recheneffizienz**: Im Vergleich zur komplexen Bildverarbeitung (Computer Vision), die hohe Anforderungen an die CPU/GPU stellt, erlauben LiDAR-Scans eine schnelle Extraktion von Hindernissen in Form von Distanz-Arrays.

3. **Beleuchtungsunabhängigkeit**: LiDAR-Systeme arbeiten als aktive Sensoren unabhängig von externen Lichtverhältnissen, was die Robustheit der Hinderniserkennung in unterschiedlichen Laborumgebungen sicherstellt.

Durch die Kombination der LiDAR-basierten Distanzmessung mit der geometrischen Logik des Follow-the-Gap-Verfahrens wird eine Architektur realisiert, die sowohl die Anforderungen an die Echtzeitfähigkeit als auch an die Kollisionssicherheit erfüllt.

1.5 Follow-the-Gap als reaktiver Planungsansatz

Der Follow-the-Gap-Algorithmus (FGM) ist ein lokaler Planungsansatz, der eine sichere Durchquerung von Umgebungen ermöglicht, ohne auf eine globale Karte angewiesen zu sein [1]. Er berechnet basierend auf der aktuellen Sensordatenlage reaktiv den nächsten Bewegungsschritt [1].

1.5.1 Grundprinzip und methodisches Vorgehen

Die Implementierung nach Sezer und Gokasan transformiert das Problem zunächst durch eine Hindernisvergrößerung (*Obstacle Enlargement*) in einen Punkt-Roboter-Ansatz [1]. Dabei wird der Radius des Fahrzeugs rechnerisch zum Radius der Hindernisse addiert, um eine kollisionsfreie Trajektorie innerhalb der berechneten Lücken zu garantieren [1].

Der Algorithmus folgt drei wesentlichen Schritten [1]:

1. **Berechnung des Gap-Arrays**: Basierend auf den Sensordaten sowie den Sichtfeldgrenzen (*Field of View*) und den kinematischen Grenzen (*Nonholonomic Constraints*) werden freie Bereiche zwischen den Hindernissen identifiziert [1].

2. **Bestimmung des Gap-Zentrums**: Statt lediglich den arithmetischen Mittelwert der Winkel zu bilden, nutzt das Verfahren geometrische Ansätze wie das Cosinus- und das Apollonius-Theorem, um den Median-Vektor zur Mitte der Lücke zu berechnen [1]. Dies sorgt für eine Trajektorie, die den Abstand zu den begrenzenden Objekten maximiert und somit die Sicherheit erhöht [1].

3. **Berechnung des finalen Kurswinkels**: Eine Fusing-Funktion kombiniert den berechneten Winkel zum Gap-Zentrum mit dem Winkel zum globalen Ziel [1].

Das System gewichtet dabei die Sicherheit (Gap) höher, wenn Hindernisse in unmittelbarer Nähe zum Fahrzeug detektiert werden, während bei freien Fahrwegen die Zielerreichung priorisiert wird [1].

1.5.2 Eigenschaften des Verfahrens

Das FGM-Verfahren zeichnet sich durch folgende Merkmale aus:

- **Minimale Parametrierung**: Die Steuerung des Verhältnisses zwischen Zielorientierung und Sicherheit erfolgt über lediglich einen zentralen Tuning-Parameter (α) [1].
- **Berücksichtigung der Kinematik**: Der Algorithmus bezieht den minimalen Wendekreis des Fahrzeugs direkt in die Lückenberechnung ein, wodurch nur physikalisch fahrbare Trajektorien generiert werden [1].
- **Vermeidung lokaler Minima**: Im Gegensatz zu klassischen Potentialfeldmethoden weist FGM aufgrund seiner geometrischen Struktur kein inhärentes Problem mit lokalen Minima auf [1].

1.5.3 Scan-basierte Umsetzung im ROS 2-Kontext

In der praktischen Umsetzung für das MXCarkit wird dieser theoretische Ansatz in eine modulare ROS 2-Softwarearchitektur überführt [7]. Die primäre Eingangsgröße bildet dabei die Nachricht `sensor_msgs/LaserScan` des LiDAR-Sensors [7]. Die Implementierung im Repository `ftg_mxck` bildet die methodischen Schritte des Prozesses als reaktive Verarbeitungskette ab [7]:

```
/scan (Rohdaten des LiDAR)
  -> Vorverarbeitung (Filterung, Beruecksichtigung der
     Fahrzeuggeometrie)
  -> FGM-Kern (Geometrische Berechnung des Gap-Zentrums)
  -> Planner (Gewichtung von Sicherheit und Zielvorgabe via Alpha)
  -> Control (Ausgabe als /autonomous/ackermann_cmd)
```

Diese Struktur ermöglicht eine stabile Integration des algorithmischen Kerns von Sezer und Gokasan in ein verteiltes System und wahrt dabei die Echtzeitfähigkeit der Hindernisvermeidung [7].

2 Systembeschreibung und Durchführung

Dieses Kapitel beschreibt die konkret umgesetzte Systemarchitektur sowie die methodische Durchführung der Implementierung und Validierung des Follow-the-Gap-Stacks im Repository `ftg_mxck`. Der Fokus liegt auf dem scan-basierten Hauptpfad, der als primäre Laufzeitarchitektur definiert wurde.

2.1 Gesamtarchitektur des Systems

Die entwickelte Pipeline folgt einer strikt modularen Struktur, die sich in vier logisch getrennte Verarbeitungsebenen gliedert:

1. LiDAR-Vorverarbeitung (Perception)
2. Reaktiver Planerkern (Follow-the-Gap)
3. Policy-basierte Entscheidungslogik (Planner)
4. Fahrzeugschnittstelle (Control)

Die resultierende Datenflusskette ist in Abbildung 2.1 dargestellt.

```
/scan (LaserScan, frame: laser)
  |
  ▼
┌──────────────────────┐
│  scan_preprocessor_node │  Paket: mxck_ftg_perception
│  TF: base_link ← laser  │  Rezentrierung auf Fahrzeug-Front
│  FOV: ±60° (120°)       │  Clipping: 0.25–10.0 m
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/scan_filtered  (LaserScan, frame: base_link)
         └──▶ /autonomous/ftg/front_clearance (Float32, Meter)
               |                                  |
               ▼                                  |
┌──────────────────────┐                          |
│    follow_the_gap_v0    │  Paket: follow_the_gap_v0│
│    (C++, input_mode=scan)│  CTU FTG-Algorithmus     |
└──────────────────────┘                          |
         |                                        |
         |                                        |
         ├──▶ /final_heading_angle (Float32, rad)    |
         ├──▶ /gap_found           (Bool)            |
         ├──▶ /visualize_obstacles (Marker)          |
         ├──▶ /visualize_largest_gap (PointStamped)  |
         └──▶ /visualize_final_heading_angle (PoseStamped)
               |                                  |
               ▼                                  ▼
┌──────────────────────────────────────────────┐
│              ftg_planner_node                  |
│  Paket: mxck_ftg_planner                       |
│  Speed-Policy: clearance + steering → Geschwindigkeit|
│  Heading-Smoothing + Speed-Smoothing           |
└──────────────────────────────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/gap_angle      (Float32, rad)
         ├──▶ /autonomous/ftg/target_speed   (Float32, m/s)
         └──▶ /autonomous/ftg/planner_status (String)
               |
               ▼
┌──────────────────────┐
│    ftg_command_node     │  Paket: mxck_ftg_control
│    → AckermannDriveStamped
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ackermann_cmd       (AckermannDriveStamped)
         └──▶ /autonomous/ftg/control_status  (String)
               |
               ▼
         [vehicle_control]  (extern, nicht Teil dieses Stacks)
```

Abbildung 2.1: Gesamtarchitektur des scan-basierten FTG-Stacks . Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die klare Trennung zwischen Wahrnehmung, Planung und Steuerung.[7]

Der vollständige Aufbau sowie alle Konfigurationsdateien sind im Repository `ftg_-mxck` dokumentiert. Im Rahmen dieser Arbeit wird auf eine vollständige Reproduktion aller Startbefehle verzichtet; stattdessen wird die Architektur konzeptionell beschrieben und auf die implementierten Komponenten verwiesen.

2.2 Algorithmische Grundlage

Der eingesetzte Planungsansatz basiert auf dem Follow-the-Gap-Verfahren nach Sezer und Gokasan [1]. Die konkrete Implementierung orientiert sich an der F1TENTH-/CTU-Referenz [3] und wurde unverändert als C++-Kern in den Stack integriert.

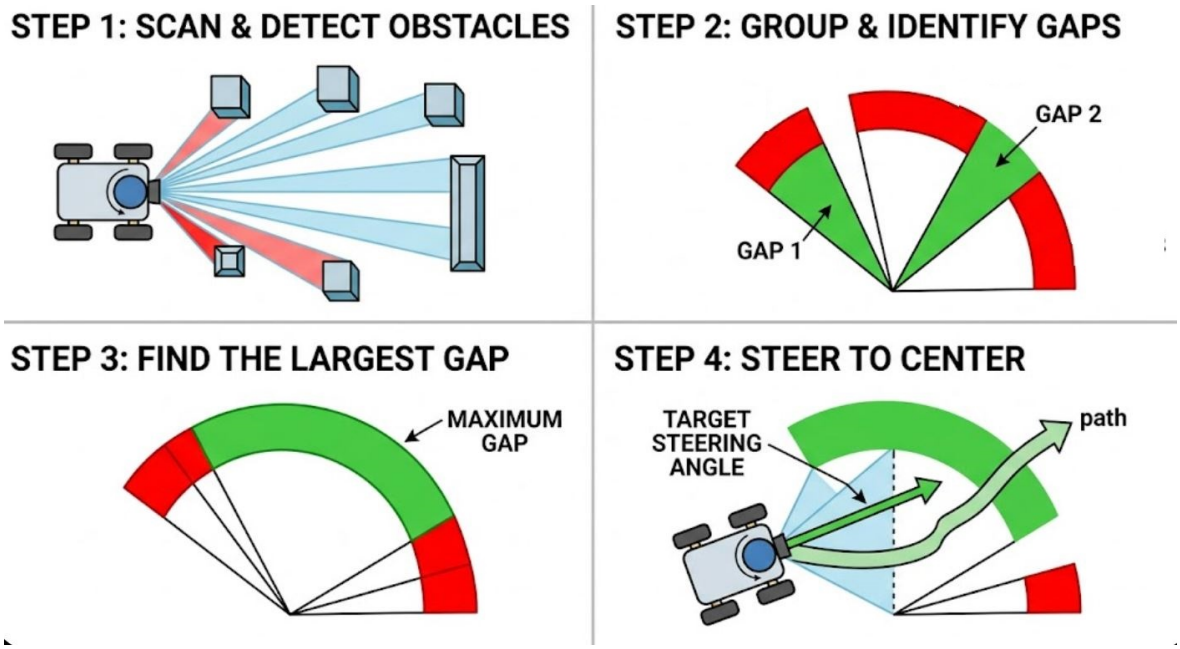

Abbildung 2.2: Schematische Darstellung des Follow-the-Gap-Prinzips [6].

Die zentrale Idee besteht darin, aus einem eindimensionalen Distanzprofil die größte kollisionsfreie Lücke zu bestimmen und das Fahrzeug entlang deren Mittelpunkt auszurichten. Diese Strategie ermöglicht eine robuste reaktive Navigation ohne globale Karteninformation.

2.3 Stufe 1: LiDAR-Vorverarbeitung

Die Vorverarbeitung transformiert den Rohscan in eine für den FTG-Kern geeignete Repräsentation.

2.3.1 Funktionale Aufgaben

Die implementierte Verarbeitung umfasst:

- Transformation des Scans in den Fahrzeugrahmen mittels TF,
- Definition eines frontalen Sichtfensters,
- Entfernung ungültiger Messwerte,
- Glättung des Signals zur Rauschreduktion,
- Berechnung des minimalen Frontabstands.

Diese Schritte sind notwendig, um die geometrische Konsistenz zwischen Sensor und Fahrzeug sicherzustellen.

2.3.2 Node Perception Erklärung

Bla Bla Bla

2.4 Stufe 2: Follow-the-Gap-Kern

Der Planerkern bildet das algorithmische Zentrum des Systems. Er verarbeitet den vorbereiteten Scan und berechnet daraus einen Richtungswinkel.

2.4.1 Verarbeitungsschritte

Innerhalb des Algorithmus erfolgen:

1. Identifikation von Hindernisbereichen,
2. Bestimmung zusammenhängender freier Bereiche (Gaps),
3. Auswahl des größten befahrbaren Gaps,
4. Berechnung eines Zielwinkels innerhalb dieses Gaps.

2.4.2 Ausgabegrößen

Der Algorithmus liefert:

- einen kontinuierlichen Heading-Winkel,
- ein binäres Signal zur Gap-Erkennung.

Diese Ausgabe ist bewusst fahrzeugagnostisch gehalten und enthält keine Informationen über Geschwindigkeit oder Dynamik.

2.5 Stufe 3: Planner

Der Planner interpretiert die algorithmische Ausgabe im Kontext der Fahrzeugdynamik und Sicherheitsanforderungen.

2.5.1 Funktionale Rolle

Im Gegensatz zum FTG-Kern handelt es sich hierbei nicht um einen Planungsalgorithmus im engeren Sinne, sondern um eine Entscheidungslogik, die folgende Aufgaben übernimmt:

- Validierung der Eingangsdaten (Freshness),

- Begrenzung des Lenkwinkels,
- adaptive Geschwindigkeitswahl in Abhängigkeit vom Frontabstand,
- Zustandsbewertung des Systems.

2.5.2 Interpretation der Sensordaten

Die Geschwindigkeit wird als Funktion des minimalen Abstands gewählt. Dabei gilt:

- große Abstände → hohe Geschwindigkeit,
- kleine Abstände → reduzierte Geschwindigkeit,
- kritische Abstände → Stopp.

Diese Policy stellt sicher, dass der reaktive Charakter des FTG-Ansatzes nicht zu unsicheren Fahrzuständen führt.

2.6 Stufe 4: Control

Die letzte Stufe des Systems übernimmt die Transformation der Planner-Ausgabe in fahrzeugspezifische Steuerbefehle.

2.6.1 Aufgabe

Die berechneten Sollwerte werden in ein Ackermann-kompatibles Nachrichtenformat überführt und an die bestehende Fahrzeugsteuerung übergeben.

2.6.2 Systemintegration

Ein wesentliches Designkriterium ist, dass die bestehende Sicherheits- und Steuerlogik des MXCarkit unverändert bleibt. Der entwickelte Stack liefert ausschließlich Sollwerte und greift nicht direkt in die Aktorik ein.

2.7 Versuchsdurchführung

Die Validierung des Systems erfolgte anhand realer Fahrversuche im Innenraum.

2.7.1 Versuchsaufbau

Die Tests wurden in einem strukturierten Korridor durchgeführt, der durch gezielt platzierte Hindernisse definiert wurde.

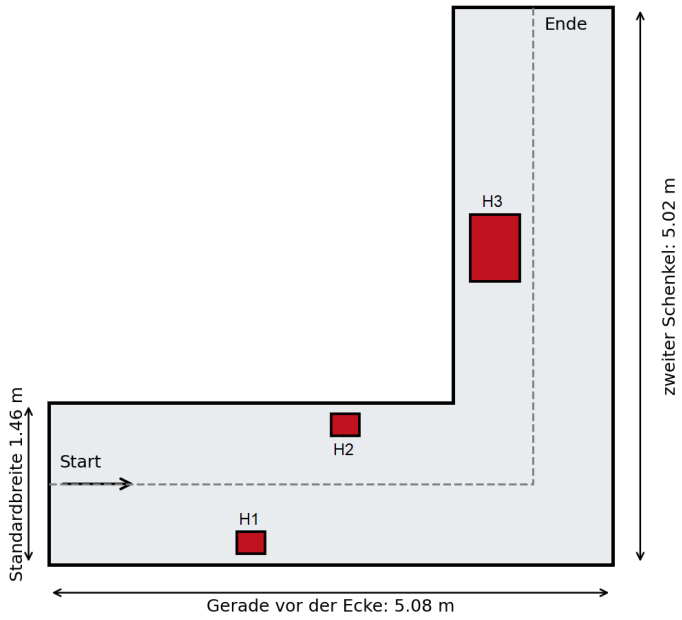

Abbildung 2.3: Versuchsaufbau Aufnahme 1: Korridor mit einem statischen Hindernis.

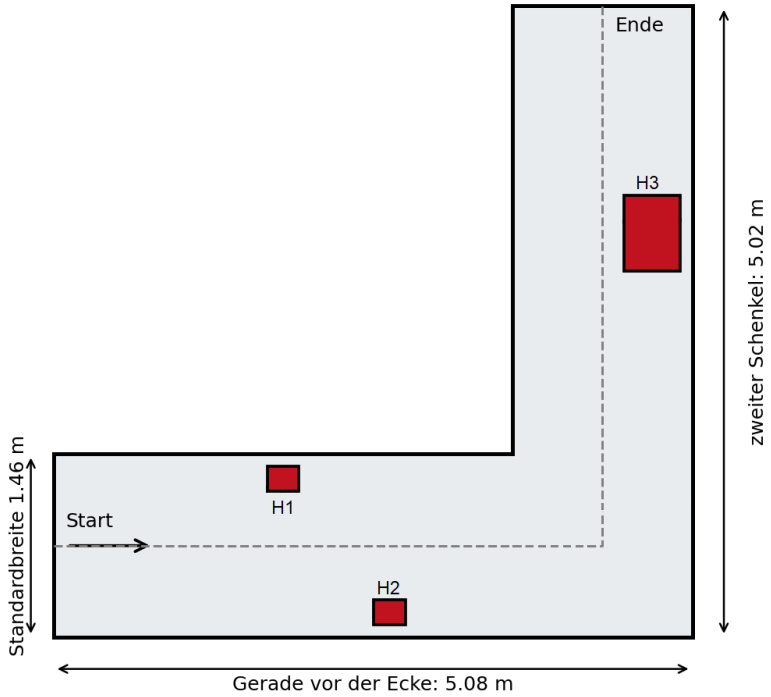

Abbildung 2.4: Versuchsaufbau Aufnahme 2: Mehrere Hindernisse und dynamische Veränderung der Umgebung.

2.7.2 Parametrierung des Systems

Die Validierungsfahrten wurden mit einer fest definierten Konfiguration des FTG-Stacks durchgeführt. Die Parametrierung ist für die Bewertung des Systemverhaltens von zentraler Bedeutung, da sie sowohl die geometrische Interpretation der Umgebung als auch die resultierende Fahrdynamik direkt beeinflusst. Insbesondere wirken sich Sichtfeld, Distanzgrenzen, Geschwindigkeitsgrenzen sowie die Abbremslogik unmittelbar auf das Ausweich- und Kurvenverhalten des Fahrzeugs aus.

Tabelle 2.1 fasst die im Validierungsbetrieb verwendeten Laufzeitparameter der drei zentralen Softwarestufen Perception, Planner und Control zusammen.

Tabelle 2.1: Verwendete Laufzeitparameter des FTG-Stacks während der Validierungsfahrten.

Stufe / Node	**Parameter**	**Wert**	**Bedeutung und Einfluss auf das Fahrverhalten**
Perception	`front_center_deg`	0.0	Definition der Fahrtrichtung im `base_link`-Frame; die physische LiDAR-Montage wird über TF berücksichtigt.
Perception	`front_fov_deg`	180.0	Breite des für FTG verwendeten Sichtfensters; größere Werte erhöhen die Umgebungsabdeckung, lassen aber seitliche Wände stärker in die Entscheidungsfindung einfließen.
Perception	`clip_min_range_m`	0.25	Untere Distanzgrenze; unterdrückt sehr nahe Punkte und mögliche Eigenreflexionen des Fahrzeugs.
Perception	`clip_max_range_m`	10.0	Obere Distanzgrenze; begrenzt die Auswertung auf den relevanten Messbereich.
Perception	`enable_moving_average`	`true`	Aktiviert eine Glättung des Distanzsignals zur Reduktion einzelner Ausreißer.
Perception	`moving_average_window`	3	Fenstergröße der Mittelung; größere Werte erhöhen die Glättung, reduzieren aber die Reaktivität.

Stufe / Node	**Parameter**	**Wert**	**Bedeutung und Einfluss auf das Fahrverhalten**
Planner	`input_timeout_sec`	0.50	Maximales Alter der Eingangsdaten; bei Überschreitung wird aus Sicherheitsgründen gestoppt.
Planner	`max_abs_gap_angle_rad`	0.45	Begrenzung des resultierenden Lenkwinkels.
Planner	`cruise_speed_mps`	0.45	Maximale Sollgeschwindigkeit bei freier Umgebung.
Planner	`min_speed_mps`	0.20	Untergrenze der Fahrgeschwindigkeit im normalen Betrieb.
Planner	`stop_speed_mps`	0.00	Sollgeschwindigkeit im Stoppfall.
Planner	`stop_clearance_m`	0.25	Kritischer Frontabstand, unterhalb dessen das Fahrzeug anhalten soll.
Planner	`caution_clearance_m`	0.50	Abstand, ab dem wieder die volle Clearance-Freigabe erreicht wird.
Planner	`steering_slowdown_start_rad`	0.40	Ab diesem Lenkwinkel wird die Geschwindigkeit schrittweise reduziert.
Planner	`steering_slowdown_full_rad`	0.70	Lenkwinkelbereich für maximale Geschwindigkeitsreduktion; bewusst größer als der maximal zulässige Gap-Winkel gewählt, um Corner-Blockaden zu vermeiden.
Planner	`heading_smoothing_alpha`	0.4	Glättungsfaktor für den Heading-Winkel; reduziert sprunghafte Richtungswechsel zwischen aufeinanderfolgenden Scans.
Planner	`speed_smoothing_alpha`	0.3	Glättungsfaktor für die Beschleunigungsphase; Bremsvorgänge bleiben davon unberührt und werden direkt umgesetzt.

Stufe / Node	**Parameter**	**Wert**	**Bedeutung und Einfluss auf das Fahrverhalten**
Control	`input_timeout_sec`	0.50	Sicherheitsgrenze für veraltete Planner-Daten; bei Überschreitung wird ein Nullkommando ausgegeben.
Control	`angle_to_steering_gain`	-1.00	Vorzeichen- und Skalierungsfaktor für die Lenkabbildung; negative Werte invertieren die physische Lenkreaktion.
Control	`max_steering_angle_rad`	0.45	Hardwarebedingte Begrenzung des Lenkwinkels.
Control	`min_speed_mps`	0.18	Untere Grenze der ausgegebenen Sollgeschwindigkeit.
Control	`max_speed_mps`	1.80	Obergrenze der ausgegebenen Sollgeschwindigkeit.

Neben den Laufzeitparametern verwendet der C++-Kern `follow_the_gap_v0` zusätzlich interne, nicht zur Laufzeit parametrierbare Geometrie- und Gewichtungskonstanten. Diese sind in Tabelle 2.2 zusammengefasst, da sie die Passierbarkeit von Lücken und damit das resultierende Fahrverhalten ebenfalls wesentlich beeinflussen.

Tabelle 2.2: Interne FTG-Konstanten des C++-Kerns `follow_the_gap_v0`.

Konstante	**Wert**	**Bedeutung**
`kCarRadius`	0.40 m	Effektiver Sicherheitsradius zur Hindernis-Aufblähung und damit maßgeblich für die als passierbar bewertete Gap-Breite.
`kTurnRadius`	0.30 m	Modellannahme für die Kurvenfähigkeit des Fahrzeugs.
`kTrackMinWidth`	0.35 m	Mindestbreite für bestimmte Corner- und Gap-Bewertungen.
`kDistanceToCorner`	0.22 m	Sicherheitsabstand bei Corner-bezogenem Verhalten.
`kGapWeightCoefficient`	100.0	Gewichtung der Gap-Richtung in der Zielrichtungsbestimmung.
`kCornerWeightCoefficient`	100.0	Gewichtung corner-spezifischer Anteile in der Zielwahl.

Die tabellarische Zusammenstellung ist für die Reproduzierbarkeit der Ergebnisse essenziell, da bereits moderate Änderungen einzelner Parameter zu deutlich unterschiedlichem Fahrverhalten führen können. Insbesondere das Sichtfeld, die Clearance-Schwellen sowie der effektive Fahrzeugradius beeinflussen, ob enge Passagen als befahrbar oder blockiert interpretiert werden.

2.7.3 Messmethodik und Datenerfassung

Zur experimentellen Bewertung des Systems wurden sämtliche relevanten Systemgrößen während der Fahrt zeitlich synchron aufgezeichnet und anschließend offline ausgewertet. Die Datenerfassung erfolgte mit `rosbag2`, sodass sowohl Rohdaten als auch Zwischen- und Ausgangsgrößen des gesamten FTG-Stacks reproduzierbar analysiert werden konnten.

Die Auswertung fokussierte sich auf vier zentrale Beobachtungsgrößen:

- **Distanzverlauf:** Charakterisierung der Umgebungssituation über den gefilterten Scan sowie den minimalen Frontabstand,
- **Heading-Winkel:** Bewertung der vom FTG-Kern bestimmten Zielrichtung und deren Weiterverarbeitung im Planner,

- **Geschwindigkeit:** Analyse der Sollgeschwindigkeit des Planners und der tatsächlich ausgegebenen Ackermann-Geschwindigkeit,
- **Gap-Erkennung:** Bewertung, ob und wie stabil freie, befahrbare Lücken detektiert wurden.

Die aufgezeichneten Topics deckten sowohl den sensorischen Eingang als auch alle relevanten Zwischenstufen des Verarbeitungsgraphen ab. Tabelle 2.3 zeigt die für die Validierung herangezogenen Nachrichtenströme.

Tabelle 2.3: Für die Validierung aufgezeichnete Topics des FTG-Stacks.

Topic	**Nachrichtentyp**	**Funktion in der Auswertung**
`/scan`	`sensor_msgs/LaserScan`	Rohdaten des LiDARs als Eingangssignal
`/autonomous/ftg/scan_filtered`	`sensor_msgs/LaserScan`	vorverarbeiteter, rezentrierter Scan für FTG
`/autonomous/ftg/front_clearance`	`std_msgs/Float32`	minimaler Frontabstand als Eingangsgröße der Speed-Policy
`/final_heading_angle`	`std_msgs/Float32`	vom FTG-Kern berechnete Zielrichtung
`/gap_found`	`std_msgs/Bool`	binäre Information über erkannte befahrbare Lücken
`/autonomous/ftg/gap_angle`	`std_msgs/Float32`	begrenzter Planner-Lenkwinkel
`/autonomous/ftg/target_speed`	`std_msgs/Float32`	vom Planner berechnete Sollgeschwindigkeit
`/autonomous/ackermann_cmd`	`ackermann_msgs/AckermannDriveStamped`	tatsächlich ausgegebener Fahrbefehl
`/autonomous/ftg/planner_status`	`std_msgs/String`	textuelle Zustandsdiagnose des Planners
`/autonomous/ftg/control_status`	`std_msgs/String`	textuelle Zustandsdiagnose der Control-Stufe
`/visualize_obstacles`	`visualization_msgs/Marker`	Visualisierung erkannter Hindernispunkte
`/visualize_largest_gap`	`geometry_msgs/PointStamped`	Visualisierung der größten detektierten Lücke
`/visualize_final_heading_angle`	`geometry_msgs/PoseStamped`	Visualisierung der resultierenden Fahrtrichtung
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	geometrische Referenz zwischen Sensor- und Fahrzeugrahmen

Zur Online-Beobachtung des Systemzustands wurde zusätzlich *Foxglove Studio* eingesetzt. Die Live-Visualisierung diente insbesondere der Plausibilitätsprüfung von Scan, Heading, Gap-Erkennung, Planner-Zuständen und Ackermann-Ausgabe während der

Testdurchführung. Abbildung 2.5 zeigt exemplarisch ein für die Validierung verwendetes Layout mit 3D-Ansicht, Statusmeldungen und zeitbasierten Signalverläufen.

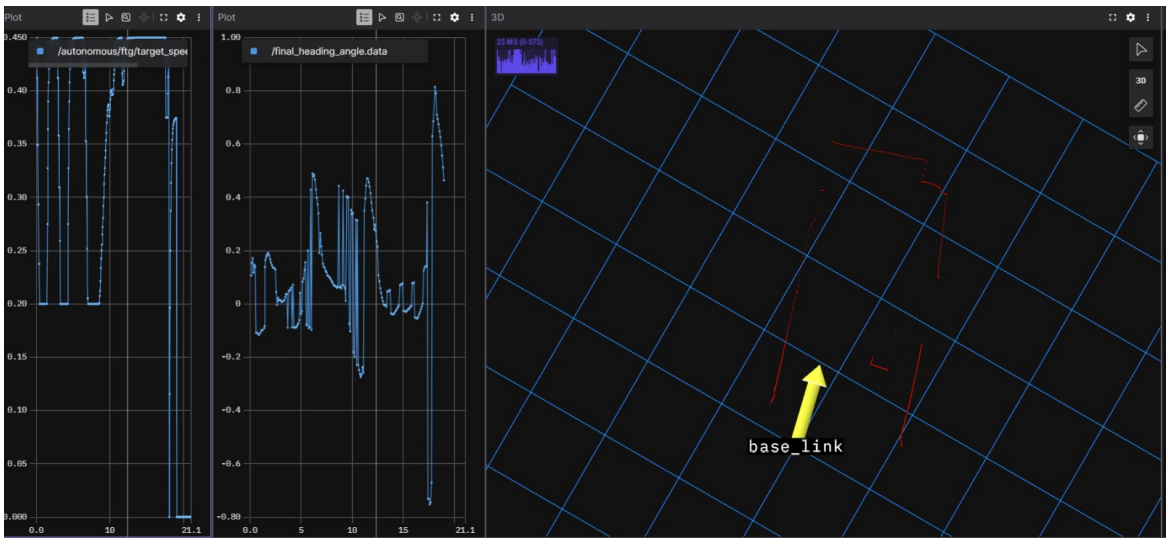

Abbildung 2.5: Exemplarisches Foxglove-Layout zur Online-Beobachtung des FTG-Stacks während der Validierungsfahrten. Dargestellt sind Sensorik, Visualisierungstopics und zentrale Zustandsgrößen des Planungs- und Steuerpfads.

Die eigentliche quantitative Bewertung erfolgte auf Basis der aufgezeichneten Bags in einer anschließenden Offline-Analyse. Hierzu wurden die Signalverläufe der beiden Validierungsläufe zeitlich ausgewertet und in Form von Metriken und Zeitreihenplots aufbereitet. Zu den wesentlichen Kenngrößen zählen unter anderem:

- mittlerer und minimaler Frontabstand,
- mittlere Sollgeschwindigkeit und Zeitanteil mit Stopp-Anforderung,
- Zeitanteil mit erfolgreicher Gap-Erkennung,
- Auftreten von Sättigungen im Heading- bzw. Gap-Winkel,
- Auftreten von Planner-Zuständen wie *normal*, *stale* oder *no valid gap found*.

Die daraus resultierenden Validierungsplots und metrischen Vergleiche werden in Kapitel 3 zur Diskussion und Bewertung der beiden Teststrecken herangezogen.

3 Diskussion und Bewertung

Dieses Kapitel bewertet die entwickelte Systemarchitektur hinsichtlich ihrer technischen Eignung, Robustheit, Reaktionsfähigkeit und Übertragbarkeit. Im Mittelpunkt stehen dabei nicht nur die grundsätzliche Funktionsfähigkeit des scan-basierten Follow-the-Gap-Stacks, sondern auch die Frage, wie zuverlässig Hindernisse bzw. freie befahrbare Bereiche erkannt werden, wie sicher die resultierenden Ausweichmanöver ausfallen und wie stark die Parametrierung das Fahrverhalten beeinflusst.

3.1 Bewertung der gewählten Gesamtarchitektur

Die Entscheidung für eine strikt scan-basierte Pipeline stellt eine bewusste Abkehr von komplexeren, mehrstufigen Wahrnehmungsansätzen dar. Anstelle einer expliziten Objektextraktion oder einer modellbasierten Hindernisrepräsentation wird der freie und belegte Raum direkt aus dem LiDAR-Signal interpretiert. Dies entspricht dem Grundgedanken des Follow-the-Gap-Verfahrens, das auf eine reaktive Auswahl befahrbarer Lücken aus dem aktuellen Sichtfeld abzielt.

3.1.1 Reduktion von Komplexität

Durch den direkten Übergang von `/scan` zu einer gefilterten Scan-Repräsentation wird auf zusätzliche Abstraktionsebenen wie Objektdetektion, Segmentierung oder Kartierung verzichtet. Dies reduziert:

- die Anzahl potenzieller Fehlerquellen innerhalb der Verarbeitungskette,
- den Parametrierungs- und Integrationsaufwand,
- die Latenz zwischen Sensormessung und Fahrreaktion.

Gerade für reaktive Verfahren wie Follow-the-Gap ist diese Vereinfachung fachlich sinnvoll, da der Algorithmus explizit für rohnahe Sensordaten formuliert wurde und seine Stärke in der schnellen lokalen Reaktion auf Hindernisse liegt.

3.1.2 Vergleich mit alternativen Ansätzen

Im Gegensatz zu globalen Planern oder kartengestützten Verfahren besitzt der implementierte Stack keine Kenntnis über die Umgebung außerhalb des aktuellen Sichtfelds. Daraus ergeben sich zwei zentrale Eigenschaften:

- eine hohe Reaktionsfähigkeit auf lokale Änderungen der Umgebung,
- jedoch eine eingeschränkte Fähigkeit zur global optimalen Pfadplanung.

Die Architektur ist daher klar als **lokaler reaktiver Planer** einzuordnen und nicht als vollständiges Navigationssystem. Diese Einordnung ist für die Bewertung wesentlich, da Einschränkungen in komplexen Geometrien nicht als Integrationsfehler, sondern als systemimmanente Folge des gewählten Planungsansatzes zu interpretieren sind.

3.2 Analyse der Planner- und Control-Schicht

Die Trennung zwischen algorithmischem Kern, Planner und Control erzeugt eine klare funktionale Hierarchie innerhalb des Systems. Der FTG-Kern liefert eine fahrzeugagnostische Richtungsinformation, während Planner und Control diese Information in fahrbare und sicherheitsgerichtete Sollwerte überführen.

Die gewählte Aufteilung ermöglicht:

- Wiederverwendbarkeit des FTG-Kerns unabhängig von der Fahrzeugplattform,
- unabhängige Anpassung der Fahrstrategie durch Parametrierung der Planner-Logik,
- klare Definition von Verantwortlichkeiten innerhalb der Pipeline.

Insbesondere fungiert der Planner als **Policy-Schicht**, die sicherstellt, dass die reaktive Planung in physikalisch realisierbare und sichere Fahrbefehle überführt wird. Die Control-Schicht bildet anschließend die Schnittstelle zur bestehenden Fahrzeugsteuerung und begrenzt die Planner-Ausgabe entsprechend der fahrzeugspezifischen Randbedingungen.

3.3 Bewertung der experimentellen Ergebnisse

Die durchgeführten Fahrversuche zeigen, dass das System in strukturierten Innenräumen grundsätzlich funktionsfähig ist, die erreichte Qualität des Fahrverhaltens jedoch deutlich von Umgebungsgeometrie und Parametrierung abhängt. Besonders relevant sind dabei die Stabilität der Gap-Erkennung, das Auftreten von Stopps bzw. Fast-Stop-Zuständen, die Qualität der Eckdurchfahrt sowie der Einfluss des Sichtfelds und der geometrischen FTG-Konstanten.

3.3.1 Quantitativer Vergleich der Validierungsläufe

Tabelle 3.1 fasst die wichtigsten Kenngrößen der beiden Validierungsläufe zusammen. Die Werte basieren auf der Offline-Auswertung der aufgezeichneten Bags und beziehen sich sowohl auf die wahrnehmungsnahen Größen als auch auf die vom Planner und von der Control-Schicht erzeugten Signale.

Tabelle 3.1: Vergleich zentraler Metriken der beiden Validierungsläufe.

Metrik	**Validierungslauf A**	**Validierungslauf B**
Gap-Erkennung (`gap_found = true`)	89,8 %	56,0 %
Planner-Zustand *normal*	90,0 %	55,5 %
Planner-Zustand *stale*	0,0 %	32,8 %
Planner-Zustand *no valid gap found*	10,0 %	11,7 %
Mittlere Sollgeschwindigkeit `target_speed`	0,335 m/s	0,226 m/s
Mittlerer Frontabstand `front_clearance`	0,601 m	0,685 m
Heading-Sättigung ($\lvert heading \rvert \geq 0{,}45$ rad)	11,4 %	–
Gap-Sättigung ($\lvert gap_angle \rvert \geq 0{,}45$ rad)	5,2 %	–
Zeitanteil mit Stopp-Anforderung (`target_speed = 0`)	gering	44,2 %
Kollisionen	0	0

Die Gegenüberstellung zeigt, dass Validierungslauf A insgesamt ein deutlich stabileres Verhalten aufweist. Lauf B ist dagegen durch einen wesentlich höheren Anteil kritischer Zustände gekennzeichnet, insbesondere durch häufige Planner-Zustände *stale* sowie einen hohen Anteil von Stopp-Anforderungen.

3.3.2 Zeitreihenbasierte Detailanalyse

Zur qualitativen Interpretation der numerischen Kennwerte wurden die wichtigsten Signalverläufe beider Fahrten zeitlich ausgewertet. Dabei wurden insbesondere Frontab-

stand, Heading-Winkel, Gap-Winkel, Sollgeschwindigkeit und Planner-Zustände herangezogen.

Abbildung 3.1: Zeitreihenbasierte Auswertung von Validierungslauf A. Dargestellt sind Frontabstand, Heading- und Gap-Winkel, Sollgeschwindigkeit sowie Zustandsgrößen des Planners.

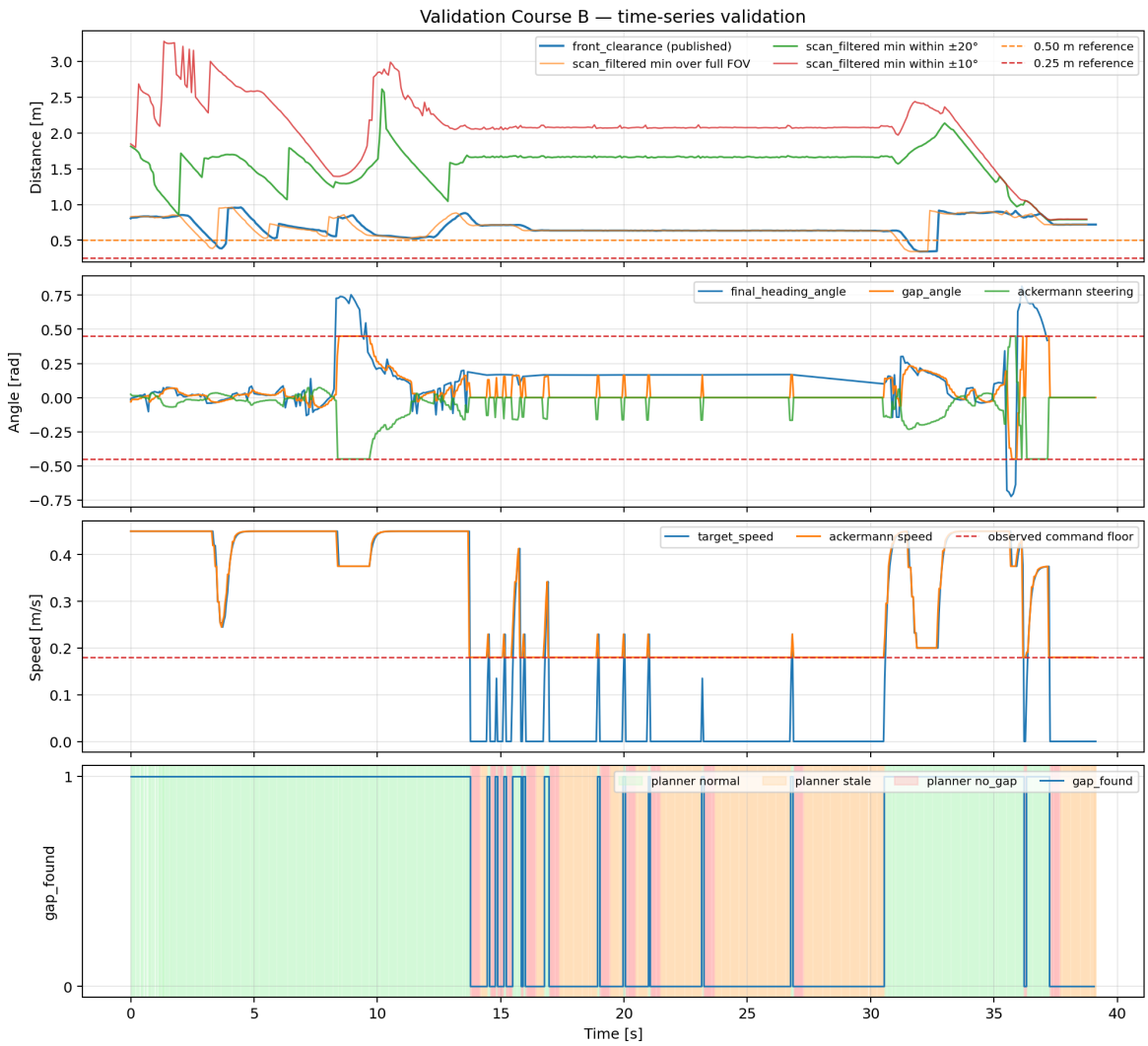

Abbildung 3.2: Zeitreihenbasierte Auswertung von Validierungslauf B. Die Darstellung verdeutlicht Unterschiede in Gap-Erkennung, Geschwindigkeitsfreigabe und Planner-Zuständen im Vergleich zu Validierungslauf A.

Die Zeitreihen zeigen, dass Lauf A über weite Strecken stabile und konsistente Reaktionen des Systems aufweist. In Lauf B treten dagegen deutlich häufiger Phasen auf, in denen keine valide Lücke erkannt wird oder der Planner auf veraltete Eingangsdaten reagiert. Dies führt zu einer größeren Zahl unterbrochener oder stark verlangsamter Fahrabschnitte.

3.3.3 Bewertung der Hinderniserkennung

Die Hinderniserkennung erfolgt im vorliegenden System nicht als semantische Objekterkennung, sondern als Interpretation freier und belegter Bereiche im LiDAR-Scan. Bewertet wird daher nicht die Klassifikation einzelner Objekte, sondern die Zuverlässigkeit, mit der der Stack befahrbare Lücken erkennt und den unmittelbaren Fahrweg

als frei oder eingeschränkt bewertet.

Die Ergebnisse zeigen:

- Die Gap-Erkennung ist in Validierungslauf A mit knapp 90 % sehr stabil.
- In Validierungslauf B sinkt die Gap-Erkennungsrate deutlich auf rund 56 %.
- Der Unterschied ist nicht primär auf geringere Frontabstände zurückzuführen, sondern auf eine instabilere Interpretation der Umgebungssituation.

Dies ist ein wichtiges Ergebnis: Eine hohe mittlere Clearance allein garantiert noch kein robustes Fahrverhalten. Entscheidend ist, ob aus der gemessenen Umgebung eine konsistente und fahrbare Lücke abgeleitet werden kann.

3.3.4 Bewertung der Ausweichmanöver

Die Qualität der Ausweichmanöver wurde anhand folgender Kriterien bewertet:

- Kollisionsfreiheit,
- erfolgreiche Passage der Hindernisse,
- Qualität der Seitenwahl an asymmetrischen Engstellen,
- Eckdurchfahrt,
- Kontinuität des Fahrverlaufs.

Beide Läufe verliefen kollisionsfrei. Dies ist ein zentraler Nachweis dafür, dass das System grundsätzlich in der Lage ist, auf statische Hindernisse sicher zu reagieren. Allerdings zeigt sich ein deutlicher Unterschied in der Qualität der Manöver:

- **Validierungslauf A:** Das Fahrzeug umfuhr alle Hindernisse und meisterte die 90°-Ecke grundsätzlich erfolgreich. Der Fahrverlauf ist als überwiegend stabil und funktional zu bewerten.
- **Validierungslauf B:** Die Reaktion blieb zwar kollisionsfrei, war jedoch deutlich defensiver und weniger stabil. Häufige Stopps bzw. Fast-Stop-Zustände und eine geringere Verfügbarkeit der Gap-Erkennung weisen auf eine eingeschränkte Robustheit in dieser Konfiguration hin.

Damit ist der prinzipielle Nachweis der Reaktionsstrategie erbracht, jedoch nicht in gleicher Güte für beide Szenarien.

3.3.5 Einfluss der Parametrierung auf das Fahrverhalten

Ein zentrales Ergebnis der Validierung ist, dass die Parametrierung des Systems den resultierenden Fahrstil unmittelbar beeinflusst. Dieser Zusammenhang betrifft sowohl die Wahrnehmungsstufe als auch Planner und Control sowie die internen FTG-Konstanten.

Einfluss des Sichtfelds

Das Frontsichtfeld bestimmt, wie breit die Umgebung in die lokale Reaktionsentscheidung einfließt. Ein zu großes Sichtfeld erhöht die Sensitivität gegenüber seitlichen Wänden und kann in engen Korridoren dazu führen, dass eigentlich passierbare Situationen als kritisch interpretiert werden. Ein reduziertes Sichtfeld führt dagegen zu einer robusteren Vorwärtsorientierung, kann jedoch seitliche Hindernisse später in die Entscheidungsfindung einbeziehen.

Einfluss der Clearance-Schwellen

Die Parameter `stop_clearance_m` und `caution_clearance_m` bestimmen, wie defensiv das Fahrzeug auf Hindernisannäherungen reagiert. Höhere Schwellen führen zu früherem Bremsen und damit zu einem konservativen Fahrstil. Niedrigere Schwellen erhöhen die Flussfähigkeit, können aber die Sicherheitsreserve verringern.

Einfluss der Steering-Slowdown-Parameter

Die Parameter `steering_slowdown_start_rad` und `steering_slowdown_full_rad` bestimmen, wie stark Kurven- bzw. Ausweichwinkel auf die Sollgeschwindigkeit wirken. Wird die maximale Geschwindigkeitsreduktion zu früh erreicht, kann dies insbesondere in Ecksituationen zu Corner-Blockaden oder unnötigen Stopps führen.

Einfluss der internen FTG-Geometrie

Besonders relevant ist der interne Parameter `kCarRadius`. Er bestimmt über die Hindernis-Aufblähung, welche Lücken als passierbar betrachtet werden. Der derzeit verwendete Wert von 0,4 m ist für ein Fahrzeug mit etwa 0,30 m Breite stark konservativ und führt dazu, dass enge, real befahrbare Passagen im Algorithmus als blockiert erscheinen.

Insgesamt zeigt sich: Die Parametrierung beeinflusst nicht nur die numerischen Signale des Systems, sondern den gesamten resultierenden Fahrstil, also etwa ob das Fahrzeug eher flüssig, defensiv, ruckartig oder blockierend reagiert.

3.3.6 Abgleich mit den Projektanforderungen

Die Aufgabenstellung fordert explizit den experimentellen Nachweis der Funktionalität, die Bewertung von Robustheit und Reaktionsfähigkeit sowie die kritische Analyse der Leistungsfähigkeit und Grenzen des Systems. Vor diesem Hintergrund ergibt sich der in Tabelle 3.2 dargestellte Abgleich.

Tabelle 3.2: Abgleich der wesentlichen Projektanforderungen mit den Validierungsergebnissen.

Anforderung	**Nachweis / Kennwert**	**Bewertung**
Zuverlässige Reaktion auf statische Hindernisse	kollisionsfreie Fahrten, erfolgreiche Umfahrung in beiden Läufen	erfüllt
Experimentelle Untersuchung unter Laborbedingungen	reproduzierbare Fahrversuche und rosbag-basierte Auswertung	erfüllt
Bewertung von Robustheit und Reaktionsfähigkeit	Vergleich zweier Strecken, Zeitreihenanalyse, Planner-Zustände	teilweise erfüllt
Modulare Integration in bestehende ROS 2-Infrastruktur	klar getrennter Pfad aus Perception, FTG, Planner und Control	erfüllt
Kritische Analyse von Leistungsfähigkeit und Grenzen	Identifikation von Corner-Blockaden, Stale-Zuständen und Parametereinflüssen	erfüllt

Damit kann festgehalten werden, dass die grundlegende Funktionalität und Integration erfolgreich nachgewiesen wurden. Die Robustheit ist jedoch noch nicht in allen Szenarien gleichermaßen gegeben und bleibt daher Gegenstand weiterer Optimierung.

3.3.7 Empfehlungen aus der Validierung

Aus den Messergebnissen ergeben sich mehrere konkrete Optimierungsempfehlungen:

1. **Reduktion des effektiven Fahrzeugradius im FTG-Kern:** Der Parameter

`kCarRadius` sollte von 0,4 m auf etwa 0,20 m reduziert bzw. künftig als konfigurierbarer ROS-Parameter umgesetzt werden.

2. **Trennung von FTG-Sichtfeld und Clearance-Sichtfeld:** Für die Berechnung von `front_clearance` sollte ein engeres Sichtfenster verwendet werden als für den eigentlichen FTG-Scan, um Seitenwände in Korridoren weniger stark zu gewichten.

3. **Überprüfung der Stop-Propagation:** Stopp-Anforderungen des Planners sollten in der Control-Stufe konsequent bis zum Ackermann-Ausgang durchgereicht werden.

4. **Further Tuning der Planner-Policy:** Die Clearance- und Steering-Schwellen sollten so gewählt werden, dass Corner-Blockaden vermieden werden, ohne die Sicherheitsreserve unangemessen zu reduzieren.

3.3.8 Robustheit gegenüber statischen Hindernissen

Die Experimente zeigen, dass das System statische Hindernisse grundsätzlich erkennen und in kollisionsfreie Ausweichmanöver überführen kann. Dies bestätigt die grundsätzliche Eignung des Stack-Aufbaus für strukturierte Innenraumumgebungen.

Die Robustheit ist jedoch nicht in jedem Szenario gleich hoch. Während Lauf A eine stabile und konsistente Reaktion zeigt, offenbart Lauf B eine höhere Sensitivität gegenüber Konfigurations- und Umgebungsunterschieden.

3.3.9 Grenzfälle und Grenzen des Systems

Trotz der positiven Ergebnisse treten typische Einschränkungen des gewählten Ansatzes auf:

- keine strategische Entscheidung in Sackgassen oder global komplexen Szenarien,
- potenziell suboptimale Wahl in asymmetrischen oder mehrdeutigen Geometrien,
- starke Abhängigkeit von Sichtfeld und Clearance-Parametrierung,
- konservatives Verhalten bei übergroß gewählter Fahrzeuggeometrie,
- mögliche Diskrepanzen zwischen Planner-Stop und tatsächlich ausgegebenem Fahrbefehl.

Diese Grenzen sind teils algorithmisch inhärent und teils durch die konkrete Implementierung und Parametrierung bedingt. Sie bilden damit zugleich die zentralen Ansatzpunkte für zukünftige Weiterentwicklungen.

3.4 Einordnung in den wissenschaftlichen Kontext

Die Kombination aus klassischem FTG-Ansatz, CTU-Referenzimplementierung und moderner ROS 2-Infrastruktur zeigt, wie etablierte lokale Reaktionsverfahren in eine aktuelle modulare Fahrzeugsoftware integriert werden können. Die Arbeit leistet dabei keinen Beitrag zur Weiterentwicklung des mathematischen FTG-Algorithmus selbst, sondern zur systematischen Integration, reproduzierbaren Umsetzung und experimentellen Bewertung eines bestehenden Ansatzes unter realen Randbedingungen.

Gerade im Kontext der Aufgabenstellung liegt der wissenschaftliche Mehrwert damit in drei Punkten:

- der technisch sauberen Einbettung eines reaktiven Planers in eine reale ROS 2-Fahrzeugarchitektur,
- der nachvollziehbaren experimentellen Validierung anhand definierter Kenngrößen,
- sowie der kritischen Analyse von Robustheit, Reaktionsfähigkeit und Grenzen des Systems.

4 Fazit und Ausblick

4.1 Fazit

Ziel dieser Projektarbeit war die Entwicklung, Integration und Validierung einer ROS 2-basierten Hinderniserkennung mit geeigneter Reaktionsstrategie auf dem MxCarkit-Modellfahrzeug. Im Mittelpunkt stand dabei nicht allein die Implementierung eines einzelnen Algorithmus, sondern der Aufbau einer vollständigen, reproduzierbaren Verarbeitungskette von der LiDAR-Sensorik bis zur fahrbaren Ackermann-Ausgabe.

Die entwickelte Kernarchitektur lautet:

`/scan` → `scan_preprocessor_node` → `/autonomous/ftg/scan_filtered` → `follow_the_gap_v0` → `ftg_planner_node` → `ftg_command_node` → `/autonomous/ackermann_cmd`

Damit wurde ein durchgängiger, modularer und nachvollziehbarer Datenpfad realisiert, der sich in die bestehende MXCK-Steuerarchitektur einfügt und die vorhandene Sicherheits- und Moduslogik nicht umgeht.

Die Validierung zeigt, dass das System unter Laborbedingungen grundsätzlich funktionsfähig ist. In beiden Validierungsläufen konnten Hindernisse kollisionsfrei umfahren und die Reaktionsstrategie des Systems experimentell nachgewiesen werden. Insbesondere die erfolgreiche Integration von Perception, FTG-Kern, Planner und Control belegt, dass ein scan-basiertes Hindernisvermeidungsverfahren auf dem MxCarkit praktisch betrieben werden kann.

Gleichzeitig hat die Auswertung deutlich gemacht, dass die Qualität des Fahrverhaltens wesentlich von der Parametrierung abhängt. Sichtfeld, Clearance-Schwellen, Steering-Slowdown und die internen geometrischen FTG-Konstanten beeinflussen unmittelbar, ob das Fahrzeug eher flüssig, defensiv, konservativ oder instabil reagiert. Die Arbeit zeigt damit nicht nur, dass der Stack prinzipiell funktioniert, sondern auch, dass reproduzierbare Validierung und kritische Parameterauswertung unverzichtbar für eine belastbare Bewertung sind.

Zusammenfassend wurden die wesentlichen Ziele der Aufgabenstellung erreicht:

- fachliche Einarbeitung in Hinderniserkennung, ROS 2 und MxCarkit-Systemarchitektur,
- modularer Entwurf und Integration eines scan-basierten Hindernisvermeidungsstacks,
- experimentelle Validierung anhand realer Fahrversuche,

- sowie kritische Analyse von Leistungsfähigkeit, Robustheit und Grenzen.

Die Arbeit leistet damit einen Beitrag zur systematischen Integration und Bewertung eines reaktiven Hindernisvermeidungsverfahrens auf einer realen ROS 2-Fahrzeugplattform.

4.2 Ausblick

Aus den Ergebnissen ergeben sich mehrere konkrete Weiterentwicklungsrichtungen.

Parametrierbare FTG-Geometrie

Die derzeit noch im C++-Kern fest kodierten Geometrieparameter, insbesondere `kCarRadius`, sollten künftig als ROS-Parameter verfügbar gemacht werden. Dies würde eine systematische Anpassung an reale Fahrzeugabmessungen ermöglichen und die experimentelle Vergleichbarkeit verschiedener Konfigurationen verbessern.

Verbesserung der Clearance-Bewertung

Für enge Korridorszenarien erscheint es sinnvoll, das für die Geschwindigkeitsbewertung genutzte Clearance-Fenster vom eigentlichen FTG-Sichtfeld zu trennen. Dadurch ließe sich der Einfluss seitlicher Wände auf die Längsdynamik verringern, ohne die Hinderniswahrnehmung des FTG-Kerns selbst einzuschränken.

Konsistentere Stop-Umsetzung

Ein weiterer wichtiger Entwicklungsschritt ist die konsequente Durchreichung von Planner-Stopps bis zur Control-Stufe und damit bis zum Ackermann-Ausgang. Gerade für die Sicherheitsvalidierung ist entscheidend, dass erkannte kritische Situationen nicht nur im Planner, sondern auch im endgültigen Fahrbefehl vollständig umgesetzt werden.

Erweiterte Validierung

Die vorliegende Arbeit bewertet das System in strukturierten Innenraumszenarien mit statischen Hindernissen. Für eine weitergehende Aussagekraft wären zusätzliche Untersuchungen sinnvoll, beispielsweise:

- systematische Parameterstudien mit mehreren Konfigurationssätzen,
- Vergleich verschiedener Sichtfelder und Clearance-Strategien,
- Validierung mit dynamischen Hindernissen,

- sowie Auswertung weiterer Fahrszenarien wie Engstellen, Sackgassen oder stärker variierende Kurvengeometrien.

Weiterentwicklung über lokale Reaktion hinaus

Langfristig kann der Stack durch zusätzliche Komponenten ergänzt werden, etwa durch globale Pfadplanung, Lokalisierung oder sensorfusionierte Umfeldwahrnehmung. Damit ließe sich die derzeit rein lokale Reaktionsstrategie um eine strategische Navigation erweitern.

Schlussbemerkung

Die wichtigste Erkenntnis dieser Arbeit ist, dass ein robuster Hindernisvermeidungsstack nicht allein durch den zugrunde liegenden Algorithmus bestimmt wird. Entscheidend ist vielmehr das Zusammenspiel aus Sensorvorverarbeitung, geometrischer Konsistenz, parametrierter Fahrstrategie, sauberer Systemintegration und nachvollziehbarer Validierung. Erst diese Gesamtheit macht aus einem theoretisch bekannten Verfahren eine praktisch nutzbare Fahrfunktion auf einer realen ROS 2-Plattform.

Literaturverzeichnis

[1] Sezer, V.; Gokasan, M.: *A Novel Obstacle Avoidance Algorithm: "Follow the Gap Method"*. Robotics and Autonomous Systems, Vol. 60, No. 9, 2012, pp. 1123–1134. DOI: 10.1016/j.robot.2012.05.021.

[2] Endler, M.: *Using ROS 2 for High-Speed Maneuvering in Autonomous Driving*. Bachelor's Thesis, Czech Technical University in Prague, 2022.

[3] Czech Technical University – Intelligent and Interactive Graphics Group (CTU-IIG): *F1TENTH Follow-the-Gap Implementation (f1t-ftg)*. Available at: `https://github.com/CTU-IIG/f1t-ftg` (Accessed: 2026-04).

[4] O'Kelly, M. et al.: *F1TENTH: An Open-Source Evaluation Environment for Continuous Control and Reinforcement Learning*. NeurIPS Workshop on Machine Learning for Autonomous Driving, 2019.

[5] Open Robotics: *ROS 2 Documentation*. Available at: `https://docs.ros.org` (Accessed: 2026-04).

[6] Gemini 3 Flash et al.: *KI-generierte Foto*. Erstellt am 12. April 2026. Software: ChatGPT und Google AI.

[7] Elalami, O. et al.: *ftg_mxck – ROS 2 Follow-the-Gap Stack for MXCarkit*. GitHub Repository. Available at: `https://github.com/OmarMElalami/ftg_mxck` (Accessed: 2026-04).

[8] MXCK Platform: *mxck2_ws – ROS 2 Workspace for MXCarkit*. Available at: `https://github.com/william-mx/mxck2_ws` (Accessed: 2026-04).